

Beszámoló

~ Kristóf

2026. augusztus 28.

1 2 site nearest neighbor klasszifikáció

Elkészült a No External Coupling (NEC) ág teljes klasszifikációja. A kódot én írtam hozzá, nem AI, így szerintem elég jó elképzelésem van arról, hogy a skálázás magasabb dimenziókra hogyan oldható meg gyorsan (sok helyen alaptól így vannak már implementálva a dolgok).

1.1 Algoritmus

A `2_site_solver.py` kódban található algoritmus a következőket csinálja:

- ▲ Tetszőleges operátorok láncát felépíti tenzorszorzatként. Egy `dict` objektumként megadott 2-site operátort rá tud tenni a lánc bármely i, j pontjaira (akkor is, ha nem egymás mellett helyezkednek el).
- ▲ Legyártja a legáltalánosabb ansatzot egy 2-site Hamilton-operátorra. Ebben a függvényben beállítható, hogy milyen *gauge* szerint szeretnénk dolgozni. A kód saját maga képes legyártani az egyes gauge-ekhez tartozó kényszerfeltételeket a paraméterekre vonatkozóan (ez azt jelenti, hogy attól függően, hogy gauge type I., vagy gauge type II. szerint dolgozunk, a legáltalánosabb esetben 13 vagy 10 paraméter szerepel az ansatzban).
- ▲ Egy kapcsoló segítségével ki-be kapcsolható bármilyen külső mezővel való 1-site kölcsönhatás - én azt oldottam meg, amikor nem volt egy sem.
- ▲ Periodikus határfeltételek szerint elkészíti a $\mathcal{Q}_2$ és $\mathcal{Q}_3$ töltéseket, majd kiszámítja a kommutátorukat. A kommutátort 0-vá téve a kiemeli a megoldandó harmadfokú polinomiális egyenleteket.
- ▲ A NEC esetben 44 nemtriviális egyenlet adódik így - a kód ezeket a polinomokat monomiáljai szerint linearizálva Gauss-eliminációt hajt végre, így 13 független egyenletre csökken a megoldandó rendszer. Ezeket az egyenleteket latexbe exportálja a kód.
- ▲ A paraméterekre megoldott polinomokat visszaalakítja lineáris operátorokká, így egy 22 tagú megoldásteret kapva.
- ▲ Írtam egy függvényrendszert, ami eléri, hogy minden megjelenő, akár tört alakú, vagy magasabb hatványokat tartalmazó mátrixelemet linearizáljon új szimbólumokat bevezetve, ezután elsőrendű deriváltakat véve, majd a mátrixot vektorizálva adódik egy vektorrendszer, aminek a span-je megadja, hogy az adott mátrixot használva milyen egyéb mátrixokat gyárthatunk le - ehhez csak a rangot kell vizsgálni. Ezzel a kód képes kiszűrni, hogy a megoldásként adódó mátrixok egymás speciális esetei-e, valamint, hogy részben- vagy teljesen függetlenek. Ezt végigiterálva a kapott megoldásokon a 22 mátrix 5 függetlenre csökkent.

1.2 Független megoldások

A következő független megoldásokat találtam a 2 site NN NEC esetben:

Solution family no. 0:

$$H_0 = \begin{bmatrix} 0 & 0 & A_{z+} & 0 \\ A_{-z} & 0 & A_{-+} & -A_{z+} \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -A_{-z} & 0 \end{bmatrix} \quad (1)$$

Solution family no. 1:

$$H_1 = \begin{bmatrix} \frac{A_{+-}}{4} + \frac{A_{-+}}{4} & 0 & -A_{z+} & 0 \\ 0 & -\frac{A_{+-}}{4} - \frac{A_{-+}}{4} & 0 & A_{z+} \\ -A_{z-} & 0 & -\frac{A_{+-}}{4} - \frac{A_{-+}}{4} & 0 \\ 0 & A_{z-} & 0 & \frac{A_{+-}}{4} + \frac{A_{-+}}{4} \end{bmatrix} \quad (2)$$

Solution family no. 2:

$$H_2 = \begin{bmatrix} 0 & 0 & -A_{z+} & 0 \\ 0 & 0 & -A_{-+} & A_{z+} \\ -A_{z-} & 0 & 0 & 0 \\ 0 & A_{z-} & 0 & 0 \end{bmatrix} \quad (3)$$

Solution family no. 3:

$$H_3 = \begin{bmatrix} 0 & 0 & A_{z+} & 0 \\ A_{-z} & 0 & -A_{zz} & -A_{z+} \\ 0 & -A_{zz} & 0 & 0 \\ 0 & 0 & -A_{-z} & 0 \end{bmatrix} \quad (4)$$

Solution family no. 4:

$$H_4 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ -\frac{2A_{+z}A_{zz}^2}{(A_{+z}+A_{z+})^2} & 0 & -\frac{2A_{+z}A_{zz}}{A_{+z}+A_{z+}} & 0 \\ -\frac{2A_{z+}A_{zz}^2}{(A_{+z}+A_{z+})^2} & -\frac{2A_{z+}A_{zz}}{A_{+z}+A_{z+}} & 0 & 0 \\ 0 & \frac{2A_{z+}A_{zz}^2}{(A_{+z}+A_{z+})^2} & \frac{2A_{+z}A_{zz}^2}{(A_{+z}+A_{z+})^2} & 0 \end{bmatrix} \quad (5)$$

2 Egyebek

Olvasgattam cikkeket, másodjára is nekikezdttem az *Integrable spin chains and cellular automata with medium range interaction* cikkekbe, ezúttal kicsit jobban átgondolva a cellular automatás részt is, most már sokkal több dolog világos volt, némi puskázással néhány tételt sikerült bizonyítanom nekem is, amik szerepelnek a cikkben.